

Lab 4.1: Explaining Phishing Detectors

AI for Cybersecurity (D7084E / D7041E), Group 6 · Kirill Silchenko (kirsil-5@student.ltu.se) · Stefanos Ntentopoulos (stente-5@student.ltu.se) · September 2026

1. Problem

A detector that says "this URL is phishing" is only useful to an analyst who can see *why* before blocking a site or warning users. We build detectors for one question, *is this web address a phishing site?*, in the three ways from Labs 1–3: **supervised** (every URL labelled), **semi-supervised** (5% labelled, pseudo-labels for the other 95%) and **self-supervised** (a network first learns from the URLs alone, then a classifier uses the 5% labels). We then explain them in two ways. **Glass boxes** (a small tree, a logistic regression) can be read directly. For the **black boxes** (forests, the self-supervised model) we use **post-hoc** explanations: SHAP and LIME. Finally we turn the two most important features into a Belief Rule-Based expert system (BRB) and check whether SHAP and LIME agree with its rules.

2. Setup

Data. Web Page Phishing Detection (Hannousse & Yahiouche, 2021): 11,430 URLs, exactly half phishing, 87 features (56 from the URL text, 24 from the page, 7 external lookups). Split 60/20/20, stratified, random_state=42: 6,858 training, 2,286 validation (to choose settings) and 2,286 test URLs (final scores only; every model is scored on the same test set). **Label budget:** 342 training URLs (5%, stratified, taken from the training set only) keep their label. The other 6,516 labels are hidden and are only used to count how many pseudo-labels were right, never for training. We report macro-F1, recall (share of phishing caught) and FAR (share of legitimate URLs wrongly flagged). Table 1 lists the models and how each is explained.

Table 1. Models and explainers.

Way of learning	Models	Explained with
Supervised, all labels	decision tree (depth 3 or 4, chosen on validation), logistic regression, Random Forest (300 trees)	tree rules, weights, SHAP TreeExplainer, LIME
Semi-supervised, 5%	forest on the 5% only (lower line); forest with 3 rounds of pseudo-labels (CUTOFF 0.9)	SHAP TreeExplainer
Self-supervised, 5%	logistic regression on the 5% (lower line); network that fills in 20% hidden values (32 hidden units) + logistic regression on the 5%	SHAP permutation explainer (seed 42)
Rules	BRB on the top two SHAP features, $3 \times 3 = 9$ rules from training data, Evidential Reasoning	the rules; LIME and SHAP KernelExplainer

3. Results

Table 2. All seven detectors on the same 2,286 test URLs.

Model	Macro-F1	Recall	ROC-AUC	FAR
tree	0.927	0.912	0.953	0.057
logreg	0.947	0.943	0.985	0.050
forest (100%)	0.965	0.968	0.994	0.038
forest (5%)	0.941	0.942	0.985	0.061
logreg (5%)	0.912	0.908	0.963	0.084
semi-supervised	0.934	0.940	0.980	0.071
self-supervised	0.918	0.922	0.970	0.087

The forest with all labels is best on every metric (Table 2). With 5% of the labels, the plain forest stays closest to it. The pseudo-labels were mostly right (98.9% of 4,265), but less so each round (99.4% → 95.6%).

Table 3. The five most important features per model: tree importance, absolute logistic-regression weight, and mean |SHAP value| for the three black boxes.

#	Tree	Logistic regression	SHAP forest (100%)	SHAP semi-supervised	SHAP self-supervised
1	google_index	longest_words_raw	google_index	google_index	google_index
2	page_rank	page_rank	page_rank	web_traffic	nb_www
3	nb_hyperlinks	phish_hints	nb_hyperlinks	page_rank	nb_hyphens
4	nb_www	google_index	web_traffic	nb_hyperlinks	domain_age
5	nb_qm	avg_word_host	nb_www	nb_www	page_rank

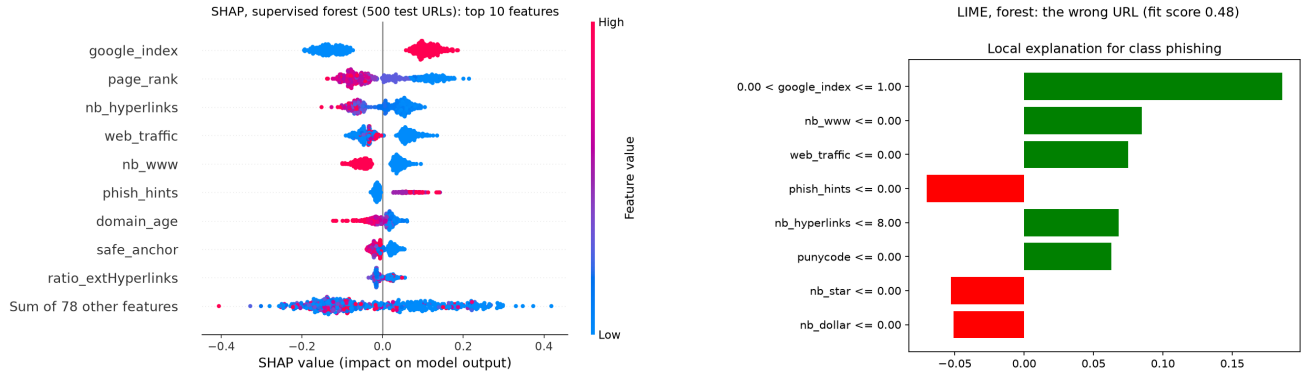


Figure 1. Left: SHAP for the supervised forest on 500 test URLs (one dot per URL; red = high value, right = toward phishing). Right: LIME for the forest's most confident mistake, graphicsfairy.blogspot.ru (legitimate, $P(\text{phishing}) = 0.950$); LIME fit score 0.481.

Table 4. Testing the detector and the explanation. Ablation: forest without the 7 external features. Stability: how many of 5 LIME runs (seeds 0–4) give the same top-3 features; SHAP gave identical values 5 of 5 times on every URL.

Test	Result
Ablation, macro-F1 / recall / FAR	with external: 0.965 / 0.968 / 0.038; without: 0.944 / 0.940 / 0.052
New top 5 without external features	nb_hyperlinks, nb_www, phish_hints, safe_anchor, ratio_extHyperlinks
LIME stability (fit score)	surest phishing URL 5 of 5 (0.62); surest legitimate 1 of 5 (0.41); the mistake 3 of 5 (0.46)

BRB. The top SHAP feature, `google_index`, is 0/1 and cannot have Low/Medium/High levels, so the antecedents are the next two: `page_rank` (how well-linked the site is) and `nb_hyperlinks` (how many links the page has). Ranking the features on 500 validation URLs instead of test URLs gives the same top 10 in the same order, so this choice does not leak test information. Referential values are the 5th percentile, median and 95th percentile of the training data (`page_rank` 0 / 3 / 8, `nb_hyperlinks` 0 / 33 / 325). Each rule's beliefs come from the share of phishing among the training URLs that activate it; the smallest rule is backed by 141 URLs (Table 5). Table 6 compares the BRB with models that see the same two features.

Table 5. The nine belief rules (beliefs in Low / Medium / High phishing risk) and their support in the training data.

Rule	page_rank	nb_hyperlinks	Low	Medium	High	Support
R1	Low	Low	0.000	0.114	0.886	1,106
R2	Low	Medium	0.000	0.318	0.682	960
R3	Low	High	0.000	0.752	0.248	141
R4	Medium	Low	0.000	0.630	0.370	854
R5	Medium	Medium	0.426	0.574	0.000	1,650
R6	Medium	High	0.837	0.163	0.000	627
R7	High	Low	0.000	0.768	0.232	344
R8	High	Medium	0.582	0.418	0.000	773
R9	High	High	0.963	0.037	0.000	403

Table 6. The BRB against models that see the same two features, and the full forest (test set).

Model	Macro-F1	Recall	ROC-AUC	FAR
BRB, 2 features	0.802	0.773	0.914	0.169
tree (depth 2), 2 features	0.803	0.948	0.885	0.334
forest, 2 features	0.847	0.842	0.927	0.148
forest, all 87 features	0.965	0.968	0.994	0.038

4. Discussion

Did the unlabelled data help? No. The semi-supervised forest is below its lower line (macro-F1 0.934 vs 0.941) and raises more false alarms (FAR 0.071 vs 0.061): the few wrong pseudo-labels cost more than the extra data gains. This does not depend on the cutoff: with 0.85, 0.90 or 0.95 no version beats the forest on the 5% on validation. The self-supervised model is only slightly above its lower line (0.918 vs 0.912), a gap that one split and one seed cannot separate from noise. As the lab warned, a forest is already strong with 342 labels.

Do the models look at the same clues? Mostly. `google_index`, `page_rank` and `nb_www` are in all five top-10 lists (Table 3 shows the top 5), and `nb_hyperlinks` and `phish_hints` in four. The tree's first question is "has Google indexed this page?", and in the beeswarm the red dots of `google_index` (1 = not indexed) are on the right: not being indexed is the forest's strongest reason to say phishing. The two forests share 8 of their top 10 features, which is expected because the pseudo-labels came from a forest. The self-supervised model shares only 5 with the forest and

adds URL-text features (`nb_hyphens`, `prefix_suffix`, `shortening_service`). Surprisingly, in its SHAP plot more hyphens push toward *legitimate*. The logistic regression is the odd one out (top weight `longest_words_raw`) and uses 79 of its 87 weights, so this "glass box" is harder to read than its top 10 suggests. The tree, with 16 leaves, really is readable, but it misses more attacks (recall 0.912).

What does the mistake show? `graphicsfairy.blogspot.ru` is a real blog on a free hosting service. It is not indexed by Google, has only 5 links, no "www" and no traffic rank: exactly the profile of a throw-away phishing page (Figure 1). SHAP and LIME agree on `google_index` and `nb_www` but share only 2 of their top 3 features, and LIME's fit score is below 0.5. On the two URLs with a fit below 0.5, LIME also changed its top 3 between seeds (Table 4), while SHAP never changed. The BRB is fooled too (phishing score 0.735): its two strongest rules, R4 and R1, both have "few links". A BRB built on a pair chosen by hand, `domain_age` + `length_url`, is not fooled (score 0.366, an old domain with a short URL), but it is much weaker overall (macro-F1 0.717). Here SHAP chose better features than intuition, but different features make different mistakes.

Which top features can an attacker fake? Everything in the URL text or the page is in the attacker's hands: adding "www" to the host name (`nb_www`), avoiding words such as "login" (`phish_hints`), or putting hundreds of hidden links on the page (`nb_hyperlinks`), which would move a phishing page into the BRB's low-risk "many links" rules. The external lookups (`google_index`, `page_rank`, `web_traffic`, `domain_age`) are much harder to fake quickly. Without them the forest's macro-F1 falls from 0.965 to 0.944 and its FAR rises from 0.038 to 0.052, and every one of its new top features is one the attacker controls.

Which model would we deploy? The forest with all labels and its SHAP explanations: it is the most accurate, SHAP TreeExplainer is exact and stable, and the analyst gets a waterfall per alert. We would run it without waiting for the live lookups as a fast first filter (it still catches 94% of phishing), and let the full model decide when the lookups arrive. We would not show LIME explanations with a fit score below 0.5. If labels are expensive, a plain forest on 5% of the labels comes close (macro-F1 0.941 against 0.965); the unlabelled data did not add anything on top of that.

The BRB and the explanations. The rules make sense: R1 (low page rank, few links) is 0.89 High risk, R9 (high page rank, many links) 0.96 Low, and risk falls when either feature grows. As the lab predicted, the BRB and a depth-2 tree have almost the same macro-F1 (0.802 vs 0.803), but the tree flags 33% of legitimate URLs against the BRB's 17%; a security team would prefer the BRB, because an alarm on every third legitimate site gets ignored. Editing R4 by hand (Medium/High → 0.2/0.8) raises recall to 0.818 but also FAR to 0.183, and makes the mistake above worse. For the mistake, SHAP agrees with the rules: the few links push High risk up most (+0.15). LIME (fit 0.74) ranks `page_rank` slightly first, because without discretisation its weights are local slopes rather than contributions. The BRB reports a large Unknown (0.315) for this URL. Checking the lab's recursive Evidential Reasoning against the analytical ER algorithm from our Lab 3 code gave identical phishing scores on all test URLs (difference below 1e-15). But the analytical version reports no Unknown, so here Unknown means "several rules fire partly" rather than "the rules disagree".

5. Who did what

[To be completed by the group: one or two lines that say who did which part of the work.]

Use of AI and sources. An AI assistant (Claude, by Anthropic, used through Claude Code) was used to plan the work, write and run the notebook code, and draft this report. We used the lab's example code, scikit-learn (Pedregosa et al., 2011), SHAP (Lundberg & Lee, 2017), LIME (Ribeiro et al., 2016), the RIMER BRB method (Yang et al., 2006), the analytical ER algorithm (Wang et al., 2006) as implemented in our Lab 3 code, and the dataset of Hannousse & Yahiouche (2021, Mendeley Data, doi:10.17632/c2gw7fy2j4.3, CC BY 4.0).

References. Hannousse, A. & Yahiouche, S. (2021). Towards benchmark datasets for machine learning based website phishing detection: An experimental study. *Engineering Applications of Artificial Intelligence*, 104, 104347. · Lundberg, S. M. & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NeurIPS* 30. · Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830. · Ribeiro, M. T., Singh, S. & Guestrin, C. (2016). "Why should I trust you?" Explaining the predictions of any classifier. *KDD 2016*. · Wang, Y.-M., Yang, J.-B. & Xu, D.-L. (2006). Environmental impact assessment using the evidential reasoning approach. *EJOR*, 174(3). · Yang, J.-B., Liu, J., Wang, J., Sii, H.-S. & Wang, H.-W. (2006). Belief rule-base inference methodology using the evidential reasoning approach (RIMER). *IEEE Trans. SMC-A*, 36(2).

Appendix: code

The notebook lab4_1_explaining_phishing_detectors.ipynb runs from top to bottom and writes every number and figure in this report. Below (Figures 2 and 3): two of its cells, the pseudo-labelling with the check of the guesses against the hidden labels (step A8), and the referential values and data-driven rules of the BRB (step D3).

```
# STEP A8
# Semi-supervised: pseudo-labelling (the same idea as Lab 2)
X_now, y_now = X_lab.copy(), y_lab.copy()          # starts with the 5% real labels
pool = X_unlab.copy()                              # URLs without a label
rf_semi = RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1)
CUTOFF = 0.9                                       # try 0.85 or 0.95 too

rounds = []                                         # one row per round, for the report
for rnd in [1, 2, 3]:
    rf_semi.fit(X_now, y_now)
    p = rf_semi.predict_proba(pool)[: , 1]
    sure = (p >= CUTOFF) | (1 - p >= CUTOFF)        # sure phishing OR sure legit
    guess = pd.Series((p[sure] >= 0.5).astype(int), index=pool.index[sure])
    X_now = pd.concat([X_now, pool[sure]])          # add them as labels
    y_now = pd.concat([y_now, guess])
    pool = pool[~sure]
    print(f"round {rnd}: added {sure.sum()} guesses,",
          f"{len(pool)} still unlabelled")

# Check only: how many of this round's guesses were right? (y_hidden is not used to train)
right = (guess == y_hidden.loc[guess.index])
rounds.append({"round": rnd, "added": int(sure.sum()),
              "added_phishing": int(guess.sum()),
              "added_legitimate": int((guess == 0).sum()),
              "correct": int(right.sum()),
              "accuracy": round(float(right.mean()), 4) if len(guess) else float("nan"),
              "still_unlabelled": len(pool)})

rf_semi.fit(X_now, y_now)                          # final model: labels + guesses
print("semi-supervised:", report(rf_semi, X_val, y_val))

rounds = pd.DataFrame(rounds)
all_guesses = y_now.iloc[len(y_lab):]              # every pseudo-label, without the 5% real ones
print(f"training rows for the final model: {len(X_now)} "
      f"{len(y_lab)} real labels + {len(all_guesses)} guesses")
print("accuracy of all guesses:",
      round(float((all_guesses == y_hidden.loc[all_guesses.index]).mean()), 4))
rounds.to_csv("results/tables/A8_pseudo_label_rounds.csv", index=False)
rounds
```

Figure 2. Step A8: pseudo-labelling; the hidden labels are only used to count correct guesses.

```

# STEP D3
# Referential values Low / Medium / High for each antecedent,
# taken from the TRAINING data: 5th percentile, median, 95th percentile
refs1 = X_train[F1].quantile([0.05, 0.50, 0.95]).round(2).tolist()
refs2 = X_train[F2].quantile([0.05, 0.50, 0.95]).round(2).tolist()
print(F1, "Low/Medium/High =", refs1)
print(F2, "Low/Medium/High =", refs2)

consequent_refs = ["Low", "Medium", "High"] # consequent: phishing risk
attribute_weights = [1.0, 1.0]
rule_weights = [1.0] * 9

# Start the 9 rules from the training data.
# For each rule: how many of the URLs that activate it are phishing?
A = np.array([calculate_activation_weights(transform_to_belief(a, refs1),
                                         transform_to_belief(b, refs2))
              for a, b in X_train[[F1, F2]].values]) # (6858 URLs, 9 rules)
support = A.sum(axis=0) # how much data per rule
phish = (A * y_train.values[:, None]).sum(axis=0) # phishing part
share = phish / np.maximum(support, 1e-9)

# Turn each phishing share into beliefs in Low / Medium / High risk
# (share 0 -> Low, 0.5 -> Medium, 1 -> High), with the same Step 1 function
rules = [transform_to_belief(s, [0.0, 0.5, 1.0]) for s in share]

names = ["Low", "Med", "High"]
for k in range(9):
    i, j = k // 3, k % 3
    print(f"R{k + 1}: {F1}={names[i]:4s} {F2}={names[j]:4s} "
          f"-> {[round(x, 2) for x in rules[k]]} (support {support[k]:.0f})")

# The rule table for the report
rule_table = pd.DataFrame({
    "rule": [f"R{k + 1}" for k in range(9)],
    F1: [{"Low", "Medium", "High"}[k // 3] for k in range(9)],
    F2: [{"Low", "Medium", "High"}[k % 3] for k in range(9)],
    "belief_Low": [round(r[0], 3) for r in rules],
    "belief_Medium": [round(r[1], 3) for r in rules],
    "belief_High": [round(r[2], 3) for r in rules],
    "phishing_share": share.round(3),
    "support": support.round(1),
})
rule_table.to_csv("results/tables/D3_rules.csv", index=False)

# A rule with (almost) no data gets share 0 and claims "Low risk" without evidence
thin = rule_table[rule_table["support"] < 20]
print("rules with support below 20: ", "none" if thin.empty else list(thin["rule"]))

# STEP D3
# Sanity checks of the engine (step D2) with these referential values
assert transform_to_belief(refs1[1], refs1) == [0.0, 1.0, 0.0] # a level value: full belief
half = (refs1[0] + refs1[1]) / 2
assert np.allclose(transform_to_belief(half, refs1), [0.5, 0.5, 0.0]) # halfway: 0.5 / 0.5
assert np.allclose(A.sum(axis=1), 1.0) # 9 weights sum to 1 per URL
b1, b2, aw, fb, ig = brbes_inference(X_train[F1].iloc[0], X_train[F2].iloc[0])
assert abs(sum(fb) + ig - 1) < 1e-9 # beliefs + Unknown = 1
print("engine sanity checks passed")

```

Figure 3. Step D3: referential values from the training data and the nine rules.